Assignment 3 - Skinning, LBS, DQS, and Correct Normals

In this assignment, you will implement linear blend skinning (LBS) and dual quaternion skinning (DQS), then use weight gradients to compute corrected normal vectors.

Getting Started

Install the assignment dependencies from the provided assignment directory:

```
conda activate comp557f26
python -m pip install -r requirements.txt
```

Run the application from that directory with `python a3_main.py`. It loads `Rectangle.glb` by default; use `python a3_main.py --file RumbaDancing.glb` to load the other provided model.

The assignment uses `pygltflib`, a library for reading glTF files containing skinned meshes and animations. Skinned animated meshes are available from sources such as Mixamo, often as FBX files that can be converted to GLB with Blender. The starter code includes two example models: a two-bone rectangle, like the one shown in class, for basic testing, and a more complex model for observing the differences between LBS and DQS. You do not need to consult the pygltflib documentation; the provided code loads the mesh and animation data and converts it to NumPy arrays. Note the assignment uses `scipy` to convert a rotation matrix to a quaternion.

Provided Code

Here is a brief overview of the provided code. Comments throughout the code offer additional guidance and warn of potential pitfalls. For instance, quaternions are often written with the real part first, but SciPy and GLSL store the real part in the last component (`w`). Take care to understand the representations used by GLM, SciPy, NumPy, and GLSL. Use Ed to discuss questions that arise.

- **`a3_main.py`** Sets up the application, loads the GLB file, creates the render context, and handles user input.
- **`asset.py`** Loads mesh data, skin weights, and joints from the GLB file and makes them available as NumPy arrays. Its `paint` method draws the skinned mesh using bone transformations

supplied by the main application. You will need to modify this file, for example, to compute weight gradients.

- **`skeleton.py`** Loads skeleton and animation data from the GLB files and computes bone transformations for a given animation time.
- **`control_panel.py`** Provides helpers for creating Qt controls in the interface.
- **`glsl/frag.glsl`** The shared fragment shader; modify it to visualize normals.
- **`glsl/vertLBS.glsl` and `glsl/vertDQS.glsl`** The vertex shaders for LBS and DQS, respectively. Modify both to implement skinning and normal correction.

Steps and Objectives

In each step, you should be able to visually verify that your implementation is working correctly.

1. **Normal Visualization (1 mark)**
   Modify the fragment shader's normal-visualization mode. Map each component of the interpolated eye-space normal from [-1, 1] to [0, 1] using `normal * 0.5 + 0.5`, so it can be displayed as RGB. A normal pointing entirely in the positive x direction should appear pastel red (`RGB = (1, 0.5, 0.5)`). Use this view to observe the problem with the naive normal transformation under pure translation in the two-bone rectangle.
2. **Implement LBS (3 marks)**
   Implement linear blend skinning in `vertLBS.glsl`. For now, transform normals using the inverse transpose of the linear part of the blended transformation (the naive approach, which you will improve later).
3. **Implement DQS (2 marks)**
   Implement dual quaternion skinning in `vertDQS.glsl`. Helper functions in `asset.py` and the vertex shader support conversion to and from dual quaternions. You can transform points and vectors using the formulas from class, or convert the blended dual quaternion back into a 4x4 transformation matrix as suggested in the comments. Include the naive normal transformation as well. When computing the blended dual quaternion, normalize all eight components using the magnitude of the real (rotation) quaternion. Ensure that all real quaternions lie in the same hemisphere: each must have a nonnegative dot product with the reference quaternion. For the provided models, you may assume the first influence has the highest weight and use its real quaternion as the reference.
4. **Compute Weight Gradients (2 marks)**
   This is the largest step. Compute and store the gradient of each bone's skinning weight at every vertex, with respect to the undeformed mesh's object-space position. The shaders include support for visualizing the weights and gradients for a selected bone. Pack the gradients correctly for

transfer to the GLSL shaders, then compare the weight and gradient visualizations on the two-bone rectangle. Adjust the gradient-visualization scale as needed.

5. **Correct Normals (2 marks)**
   Use the weight gradients to compute corrected normals for the skinned mesh in both the LBS and DQS shaders (one mark each).

Finished?

Submit a ZIP archive to MyCourses containing all Python files and GLSL shaders in a glsl subdirectory, along with your README.md. Do not include unnecessary files such as `__pycache__`, `.vscode`, this specification, or the GLB data files. Replace the name and student-number placeholders in `a3_main.py` and the README.

DOUBLE CHECK your submitted files by downloading them. You will not receive marks for assignments if files are missing, if it is a different assignment, or if the files are corrupt!

Recall the readme can be used to request a late penalty waiver following the mechanism described in the first lecture slides.

Note that you are encouraged to discuss assignments with your classmates, but not to the point of sharing code and answers. All code and written answers must be your own. Please see the course outline and the fine print in the slides of the first lecture.